

不同的速度变化着。特别地, DRAM 和磁盘访问时间远远大于 CPU 周期时间。系统通过将存储器组织成存储设备的层次结构来弥补这些差异, 在这个层次结构中, 较小、较快的设备在顶部, 较大、较慢的设备在底部。因为编写良好的程序有好的局部性, 大多数数据都可以从较高层得到服务, 结果就是存储系统能以较高层的速度运行, 但却有较低层的成本和容量。

程序员可以通过编写有良好空间和时间局部性的程序来显著地改进程序的运行时间。利用基于 SRAM 的高速缓存存储器特别重要。主要从高速缓存取数据的程序能比主要从内存取数据的程序运行得快得多。

参考文献说明

内存和磁盘技术变化得很快。根据我们的经验, 最好的技术信息来源是制造商维护的 Web 页面。像 Micron、Toshiba 和 Samsung 这样的公司, 提供了丰富的当前有关内存设备的技术信息。Seagate 和 Western Digital 的页面也提供了类似的有关磁盘的有用信息。

关于电路和逻辑设计的教科书提供了关于内存技术的详细信息[58, 89]。IEEE Spectrum 出版了一系列有关 DRAM 的综述文章[55]。计算机体系结构国际会议(ISCA)和高性能计算机体系结构(HPCA)是关于 DRAM 存储性能特性的公共论坛[28, 29, 18]。

Wilkes 写了第一篇关于高速缓存存储器的论文[117]。Smith 写了一篇经典的综述[104]。Przybylski 编写了一本关于高速缓存设计的权威著作[86]。Hennessy 和 Patterson 提供了对高速缓存设计问题的全面讨论[46]。Levinthal 写了一篇有关 Intel Core i7 的全面性能指南[70]。

Stricker 在[112]中介绍了存储器山的思想, 作为对存储器系统的全面描述, 并且在后来的工作描述中非正式地提出了术语“存储器山”。编译器研究者通过自动执行我们在 6.6 节中讨论过的那些手工代码转换来增加局部性[22, 32, 66, 72, 79, 87, 119]。Carter 和他的同事们提出了一个高速缓存可知晓的内存控制器(cache-aware memory controller)[17]。其他的研究者开发出了高速缓存不知晓的(cache oblivious)算法, 它被设计用来在不明确知道底层高速缓存存储器结构的情况下也能运行得很好[30, 38, 39, 9]。

关于构造和使用磁盘存储设备也有大量的论著。许多存储技术研究者找寻方法, 将单个的磁盘集成成更大、更健壮和更安全的存储池[20, 40, 41, 83, 121]。其他研究者找寻利用高速缓存和局部性来改进磁盘访问性能的方法[12, 21]。像 Exokernel 这样的系统提供了更多的对磁盘和存储器资源的用户级控制[57]。像安德鲁文件系统[78]和 Coda[94]这样的系统, 将存储器层次结构扩展到了计算机网络和移动笔记本电脑。Schindler 和 Ganger 开发了一个有趣的工具, 它能自动描述 SCSI 磁盘驱动器的构造和性能[95]。研究者正在研究构造和使用基于闪存的 SSD 的技术[8, 81]。

家庭作业

- 6.22 假设要求你设计一个每条磁道位数固定的旋转磁盘。你知道每条磁道的位数是由最里层磁道的周长决定的, 可以假设它就是中间那个圆洞的周长。因此, 如果你把磁盘中间的洞做得大一点, 每条磁道的位数就会增大, 但是总的磁道数会减少。如果用 r 来表示盘面的半径, $x \cdot r$ 表示圆洞的半径, 那么 x 取什么值能使这个磁盘的容量最大?
- 6.23 估计访问下面这个磁盘上扇区的平均时间(以 ms 为单位):

参数	值
旋转速率	15 000RPM
$T_{\text{avg seek}}$	4 ms
平均扇区数/磁道	800

- 6.24 假设一个 2MB 的文件, 由 512 个字节的逻辑块组成, 存储在具有下述特性的磁盘驱动器上:

参数	值
旋转速率	15 000RPM
$T_{\text{avg seek}}$	4 ms
平均扇区数/磁道	1000
盘面数	8
扇区大小	512字节

对于下面的每种情况，假设程序顺序地读文件的逻辑块，一个接一个，并且对第一个块定位读/写头的时间等于 $T_{\text{avg seek}} + T_{\text{avg rotation}}$ 。

A. 最好情况：估计在所有可能的逻辑块到磁盘扇区的映射上读该文件所需要的最优时间（以 ms 为单位）。

B. 随机情况：估计如果块是随机映射到磁盘扇区上时读该文件所需要的时间（以 ms 为单位）。

- 6.25 下面的表给出了一些不同的高速缓存的参数。对于每个高速缓存，填写出表中缺失的字段。记住 m 是物理地址的位数， C 是高速缓存大小（数据字节数）， B 是以字节为单位的块大小， E 是相联度， S 是高速缓存组数， t 是标记位数， s 是组索引位数，而 b 是块偏移位数。

高速缓存	m	C	B	E	S	t	s	b
1.	32	1024	4	4				
2.	32	1024	4	256				
3.	32	1024	8	1				
4.	32	1024	8	128				
5.	32	1024	32	1				
6.	32	1024	32	4				

- 6.26 下面的表给出了一些不同的高速缓存的参数。你的任务是填写出表中缺失的字段。记住 m 是物理地址的位数， C 是高速缓存大小（数据字节数）， B 是以字节为单位的块大小， E 是相联度， S 是高速缓存组数， t 是标记位数， s 是组索引位数，而 b 是块偏移位数。

高速缓存	m	C	B	E	S	t	s	b
1.	32		8	1		21	8	3
2.	32	2048			128	23	7	2
3.	32	1024	2	8	64			1
4.	32	1024		2	16	23	4	

- 6.27 这个问题是关于练习题 6.12 中的高速缓存的。

- 列出所有会在组 1 中命中的十六进制内存地址。
- 列出所有会在组 6 中命中的十六进制内存地址。

- 6.28 这个问题是关于练习题 6.12 中的高速缓存的。

- 列出所有会在组 2 中命中的十六进制内存地址。
- 列出所有会在组 4 中命中的十六进制内存地址。
- 列出所有会在组 5 中命中的十六进制内存地址。
- 列出所有会在组 7 中命中的十六进制内存地址。

- 6.29 假设我们有一个具有如下属性的系统：

- 内存是字节寻址的。
- 内存访问是对 1 字节字的（而不是 4 字节字）。
- 地址宽 12 位。
- 高速缓存是两路组相联的（ $E=2$ ），块大小为 4 字节（ $B=4$ ），有 4 个组（ $S=4$ ）。

高速缓存的内容如下，所有的地址、标记和值都以十六进制表示：

组索引	标记	有效位	字节0	字节1	字节2	字节3
0	00	1	40	41	42	43
	83	1	FE	97	CC	D0
1	00	1	44	45	46	47
	83	0	—	—	—	—
2	00	1	48	49	4A	4B
	40	0	—	—	—	—
3	FF	1	9A	C0	03	FF
	00	0	—	—	—	—

A. 下面的图给出了一个地址的格式(每个小框表示一位)。指出用来确定下列信息的字段(在图中标号出来):

CO 高速缓存块偏移

CI 高速缓存组索引

CT 高速缓存标记

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 对于下面每个内存访问,当它们是按照列出来的顺序执行时,指出是高速缓存命中还是不命中。如果可以从高速缓存中的信息推断出来,请也给出读出的值。

操作	地址	命中?	读出的值(或者未知)
读	0x834		
写	0x836		
读	0xFFD		

*6.30 假设我们有一个具有如下属性的系统:

- 内存是字节寻址的。
- 内存访问是对1字节字的(而不是4字节字)。
- 地址宽13位。
- 高速缓存是四路组相联的($E=4$),块大小为4字节($B=4$),有8个组($S=8$)。

考虑下面的高速缓存状态。所有的地址、标记和值都以十六进制表示。每组有4行,索引列包含组索引。标记列包含每一行的标记值。V列包含每一行的有效位。字节0~3列包含每一行的数据,标号从左向右,字节0在左边。

4路组相联高速缓存

索引	标记	V	字节0~3	标记	V	字节0~3	标记	V	字节0~3	标记	V	字节0~3
0	F0	1	ED 32 0A A2	8A	1	BF 80 1D FC	14	1	EF 09 86 2A	BC	0	25 44 6F 1A
1	BC	0	03 3E CD 38	A0	0	16 7B ED 5A	BC	1	8E 4C DF 18	E4	1	FB B7 12 02
2	BC	1	54 9E 1E FA	B6	1	DC 81 B2 14	00	0	B6 1F 7B 44	74	0	10 F5 B8 2E
3	BE	0	2F 7E 3D A8	C0	1	27 95 A4 74	C4	0	07 11 6B D8	BC	0	C7 B7 AF C2
4	7E	1	32 21 1C 2C	8A	1	22 C2 DC 34	BC	1	BA DD 37 D8	DC	0	E7 A2 39 BA
5	98	0	A9 76 2B EE	54	0	BC 91 D5 92	98	1	80 BA 9B F6	BC	1	48 16 81 0A
6	38	0	5D 4D F7 DA	BC	1	69 C2 8C 74	8A	1	A8 CE 7F DA	38	1	FA 93 EB 48
7	8A	1	04 2A 32 6A	9E	0	B1 86 56 0E	CC	1	96 3D 47 F2	BC	1	F8 1D 42 30

A. 这个高速缓存的大小(C)是多少字节?

B. 下面的图给出了一个地址的格式(每个小框表示一位)。指出用来确定下列信息的字段(在图中标号出来):

CO 高速缓存块偏移

CI 高速缓存组索引

CT 高速缓存标记

12	11	10	9	8	7	6	5	4	3	2	1	0

**** 6.31** 假设程序使用作业 6.30 中的高速缓存，引用位于地址 0x071A 处的 1 字节字。用十六进制表示出它所访问的高速缓存条目，以及返回的高速缓存字节值。指明是否发生了高速缓存不命中。如果有高速缓存不命中，对于“返回的高速缓存字节”输入“—”。提示：注意那些有效位！

A. 地址格式(每个小框表示一位)：

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 内存引用：

参数	值
高速缓存块偏移 (CO)	0x_____
高速缓存组索引 (CI)	0x_____
高速缓存标记 (CT)	0x_____
高速缓存命中? (是/否)	
返回的高速缓存字节	0x_____

**** 6.32** 对于内存地址 0x16E8 重复作业 6.31。

A. 地址格式(每个小框表示一位)：

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 内存引用：

参数	值
高速缓存块偏移 (CO)	0x_____
高速缓存组索引 (CI)	0x_____
高速缓存标记 (CT)	0x_____
高速缓存命中? (是/否)	
返回的高速缓存字节	0x_____

**** 6.33** 对于作业 6.30 中的高速缓存，列出会在组 2 中命中的 8 个内存地址(以十六进制表示)。

**** 6.34** 考虑下面的矩阵转置函数：

```

1  typedef int array[4][4];
2
3  void transpose2(array dst, array src)
4  {
5      int i, j;
6
7      for (i = 0; i < 4; i++) {
8          for (j = 0; j < 4; j++) {
9              dst[j][i] = src[i][j];
10         }
11     }
12 }
```

假设这段代码运行在一台具有如下属性的机器上：

● sizeof(int)==4。

- 数组 `src` 从地址 0 开始，而数组 `dst` 从地址 64 开始(十进制)。
 - 只有一个 L1 数据高速缓存，它是直接映射、直写、写分配的，块大小为 16 字节。
 - 这个高速缓存总共有 32 个数据字节，初始为空。
 - 对 `src` 和 `dst` 数组的访问分别是读和写不命中的唯一来源。
- 对于每个 `row` 和 `col`，指明对 `src[row][col]` 和 `dst[row][col]` 的访问是命中(h)还是不命中(m)。
- 例如，读 `src[0][0]` 会不命中，而写 `dst[0][0]` 也会不命中。

		dst数组						src数组			
		列0	列1	列2	列3			列0	列1	列2	列3
行0		m				行0		m			
行1						行1					
行2						行2					
行3						行3					

.. 6.35 对于一个总大小为 128 数据字节的高速缓存，重复练习题 6.34。

		dst数组						src数组			
		列0	列1	列2	列3			列0	列1	列2	列3
行0		m				行0		m			
行1						行1					
行2						行2					
行3						行3					

.. 6.36 这道题测试你预测 C 语言代码的高速缓存行为的能力。对下面这段代码进行分析：

```

1  int x[2][128];
2  int i;
3  int sum = 0;
4
5  for (i = 0; i < 128; i++) {
6      sum += x[0][i] * x[1][i];
7  }
```

假设我们在下列条件下执行这段代码：

- `sizeof(int)==4`。
- 数组 `x` 从内存地址 `0x0` 开始，按照行优先顺序存储。
- 在下面每种情况中，高速缓存最开始时都是空的。
- 唯一的内存访问是对数组 `x` 的条目进行访问。其他所有的变量都存储在寄存器中。

给定这些假设，估计下列情况中的不命中率：

- 情况 1：假设高速缓存是 512 字节，直接映射，高速缓存块大小为 16 字节。不命中率是多少？
- 情况 2：如果我们把高速缓存的大小翻倍到 1024 字节，不命中率是多少？
- 情况 3：现在假设高速缓存是 512 字节，两路组相联，使用 LRU 替换策略，高速缓存块大小为 16 字节。不命中率是多少？
- 对于情况 3，更大的高速缓存大小会帮助降低不命中率吗？为什么能或者为什么不能？
- 对于情况 3，更大的块大小会帮助降低不命中率吗？为什么能或者为什么不能？

.. 6.37 这道题也是测试你分析 C 语言代码的高速缓存行为的能力。假设我们在下列条件下执行图 6-47 中的 3 个求和函数：

- `sizeof(int)==4`。
- 机器有 4KB 直接映射的高速缓存，块大小为 16 字节。
- 在两个循环中，代码只对数组数据进行内存访问。循环索引和值 `sum` 都存放在寄存器中。
- 数组 `a` 从内存地址 `0x08000000` 处开始存储。

对于 $N=64$ 和 $N=60$ 两种情况，在表中填写它们大概的高速缓存不命中率。

函数	N=64	N=60
sumA		
sumB		
sumC		

```

1  typedef int array_t[N][N];
2
3  int sumA(array_t a)
4  {
5      int i, j;
6      int sum = 0;
7      for (i = 0; i < N; i++)
8          for (j = 0; j < N; j++) {
9              sum += a[i][j];
10         }
11     return sum;
12 }
13
14 int sumB(array_t a)
15 {
16     int i, j;
17     int sum = 0;
18     for (j = 0; j < N; j++)
19         for (i = 0; i < N; i++) {
20             sum += a[i][j];
21         }
22     return sum;
23 }
24
25 int sumC(array_t a)
26 {
27     int i, j;
28     int sum = 0;
29     for (j = 0; j < N; j+=2)
30         for (i = 0; i < N; i+=2) {
31             sum += (a[i][j] + a[i+1][j]
32                  + a[i][j+1] + a[i+1][j+1]);
33         }
34     return sum;
35 }

```

图 6-47 作业 6.37 中引用的函数

- * 6.38 3M 决定在白纸上印黄方格，做成 Post-It 小贴纸。在打印过程中，他们需要设置方格中每个点的 CMYK(蓝色，红色，黄色，黑色)值。3M 雇佣你判定下面算法在一个具有 2048 字节、直接映射、块大小为 32 字节的数据高速缓存上的效率。有如下定义：

```

1  struct point_color {
2      int c;
3      int m;
4      int y;
5      int k;
6  };
7
8  struct point_color square[16][16];
9  int i, j;

```

有如下假设：

- `sizeof(int)==4`。
- `square` 起始于内存地址 0。
- 高速缓存初始为空。
- 唯一的内存访问是对于 `square` 数组中的元素。变量 `i` 和 `j` 存放在寄存器中。

确定下列代码的高速缓存性能：

```

1   for (i = 0; i < 16; i++){
2       for (j = 0; j < 16; j++) {
3           square[i][j].c = 0;
4           square[i][j].m = 0;
5           square[i][j].y = 1;
6           square[i][j].k = 0;
7       }
8   }
```

- 写总数是多少？
- 在高速缓存中不命中的写总数是多少？
- 不命中率是多少？

• 6.39 给定作业 6.38 中的假设，确定下列代码的高速缓存性能：

```

1   for (i = 0; i < 16; i++){
2       for (j = 0; j < 16; j++) {
3           square[j][i].c = 0;
4           square[j][i].m = 0;
5           square[j][i].y = 1;
6           square[j][i].k = 0;
7       }
8   }
```

- 写总数是多少？
- 在高速缓存中不命中的写总数是多少？
- 不命中率是多少？

• 6.40 给定作业 6.38 中的假设，确定下列代码的高速缓存性能：

```

1   for (i = 0; i < 16; i++){
2       for (j = 0; j < 16; j++) {
3           square[i][j].y = 1;
4       }
5   }
6   for (i = 0; i < 16; i++){
7       for (j = 0; j < 16; j++) {
8           square[i][j].c = 0;
9           square[i][j].m = 0;
10          square[i][j].k = 0;
11      }
12  }
```

- 写总数是多少？
- 在高速缓存中不命中的写总数是多少？
- 不命中率是多少？

• 6.41 你正在编写一个新的 3D 游戏，希望能名利双收。现在正在写一个函数，使得在画下一帧之前先清空屏幕缓冲区。工作的屏幕是 640×480 像素数组。工作的机器有一个 64KB 直接映射高速缓存，每行 4 个字节。使用下面的 C 语言数据结构：

```

1   struct pixel {
2       char r;
3       char g;
```

```

4      char b;
5      char a;
6  };
7
8  struct pixel buffer[480][640];
9  int i, j;
10 char *cptr;
11 int *iptr;

```

有如下假设:

- `sizeof(char)==1` 和 `sizeof(int)==4`。
- `buffer` 起始于内存地址 0。
- 高速缓存初始为空。
- 唯一的内存访问是对于 `buffer` 数组中元素的访问。变量 `i`、`j`、`cptr` 和 `iptr` 存放在寄存器中。

下面代码中百分之多少的写会在高速缓存中不命中?

```

1      for (j = 0; j < 640; j++) {
2          for (i = 0; i < 480; i++){
3              buffer[i][j].r = 0;
4              buffer[i][j].g = 0;
5              buffer[i][j].b = 0;
6              buffer[i][j].a = 0;
7          }
8      }

```

** 6.42 给定作业 6.41 中的假设, 下面代码中百分之多少的写会在高速缓存中不命中?

```

1      char *cptr = (char *) buffer;
2      for (; cptr < (((char *) buffer) + 640 * 480 * 4); cptr++)
3          *cptr = 0;

```

** 6.43 给定作业 6.41 中的假设, 下面代码中百分之多少的写会在高速缓存中不命中?

```

1      int *iptr = (int *)buffer;
2      for (; iptr < ((int *)buffer + 640*480); iptr++)
3          *iptr = 0;

```

** 6.44 从 CS:APP 的网站上下载 `mountain` 程序, 在你最喜欢的 PC/Linux 系统上运行它。根据结果估计你系统上的高速缓存的大小。

** 6.45 在这项任务中, 你会把在第 5 章和第 6 章中学习到的概念应用到一个内存使用频繁的代码的优化问题上。考虑一个复制并转置一个类型为 `int` 的 $N \times N$ 矩阵的过程。也就是, 对于源矩阵 S 和目的矩阵 D , 我们要将每个元素 $s_{i,j}$ 复制到 $d_{j,i}$ 。只用一个简单的循环就能实现这段代码:

```

1 void transpose(int *dst, int *src, int dim)
2 {
3     int i, j;
4
5     for (i = 0; i < dim; i++)
6         for (j = 0; j < dim; j++)
7             dst[j*dim + i] = src[i*dim + j];
8 }

```

这里, 过程的参数是指向目的矩阵(`dst`)和源矩阵(`src`)的指针, 以及矩阵的大小 $N(\text{dim})$ 。你的工作是设计一个运行得尽可能快的转置函数。

** 6.46 这是练习题 6.45 的一个有趣的变体。考虑将一个有向图 g 转换成它对应的无向图 g' 。图 g' 有一条从顶点 u 到顶点 v 的边, 当且仅当原图 g 中有一条 u 到 v 或者 v 到 u 的边。图 g 是由如下的它的邻接矩阵(adjacency matrix) G 表示的。如果 N 是 g 中顶点的数量, 那么 G 是一个 $N \times N$ 的矩阵, 它的元素是全 0 或者全 1。假设 g 的顶点是这样命名的: $v_0, v_1, \dots, v_{N-1}$ 。那么如果有一条从 u_i 到 v_j 的边, 那么 $G[i][j]$ 为 1, 否则为 0。注意, 邻接矩阵对角线上的元素总是 1, 而无向图的邻

接矩阵是对称的。只用一个简单的循环就能实现这段代码：

```
1 void col_convert(int *G, int dim) {
2     int i, j;
3
4     for (i = 0; i < dim; i++)
5         for (j = 0; j < dim; j++)
6             G[j*dim + i] = G[j*dim + i] || G[i*dim + j];
7 }
```

你的工作是设计一个运行得尽可能快的函数。同前面一样，要提出一个好的解答，你需要应用在第5章和第6章中所学到的概念。

练习题答案

- 6.1 这里的思想是通过使纵横比 $\max(r, c)/\min(r, c)$ 最小，使得地址位数最小。换句话说，数组越接近于正方形，地址位数越少。

组织	r	c	b_r	b_c	$\max(b_r, b_c)$
16×1	4	4	2	2	2
16×4	4	4	2	2	2
128×8	16	8	4	3	4
512×4	32	16	5	4	5
1024×4	32	32	5	5	5

- 6.2 这个小练习的主旨是确保你理解柱面和磁道之间的关系。一旦你弄明白了这个关系，那问题就很简单了：

$$\begin{aligned}\text{磁盘容量} &= \frac{512 \text{ 字节}}{\text{扇区}} \times \frac{400 \text{ 扇区数}}{\text{track}} \times \frac{10\,000 \text{ 磁道数}}{\text{表面}} \times \frac{2 \text{ 表面数}}{\text{盘片}} \times \frac{2 \text{ 盘片数}}{\text{磁盘}} \\ &= 8\,192\,000\,000 \text{ 字节} \\ &= 8.192\text{GB}\end{aligned}$$

- 6.3 对这个问题的解答是对磁盘访问时间公式的直接应用。平均旋转时间(以 ms 为单位)为

$$T_{\text{avg rotation}} = 1/2 \times T_{\text{max rotation}} = 1/2 \times (60\text{s}/15\,000\text{RPM}) \times 1000\text{ms/s} \approx 2\text{ms}$$

平均传送时间为

$$T_{\text{avg transfer}} = (60\text{s}/15\,000\text{RPM}) \times 1/500 \text{ 扇区} / \text{磁道} \times 1000\text{ms/s} \approx 0.008\text{ms}$$

总的来说，总的预计访问时间为

$$T_{\text{access}} = T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}} = 8\text{ms} + 2\text{ms} + 0.008\text{ms} \approx 10\text{ms}$$

- 6.4 这道题很好的检查了你对影响磁盘性能的因素的理解。首先我们需要确定这个文件和磁盘的一些基本属性。这个文件由 2000 个 512 字节的逻辑块组成。对于磁盘， $T_{\text{avg seek}} = 5\text{ms}$ ， $T_{\text{max rotation}} = 6\text{ms}$ ，而 $T_{\text{avg rotation}} = 3\text{ms}$ 。

A. 最好情况：在好的情况中，块被映射到连续的扇区，在同一柱面上，那样就可以一块接一块地读，不用移动读/写头。一旦读/写头定位到了第一个扇区，需要磁盘转两整圈(每圈 1000 个扇区)来读所有 2000 个块。所以，读这个文件的总时间为 $T_{\text{avg seek}} + T_{\text{avg rotation}} + 2 \times T_{\text{max rotation}} = 5 + 3 + 12 = 20\text{ms}$ 。

B. 随机的情况：在这种情况下，块被随机地映射到扇区上，读 2000 块中的每一块都需要 $T_{\text{avg seek}} + T_{\text{avg rotation}}$ ms，所以读这个文件的总时间为 $(T_{\text{avg seek}} + T_{\text{avg rotation}}) \times 2000 = 16\,000\text{ms}$ (16 秒!)

你现在可以看到为什么清理磁盘碎片是个好主意！

- 6.5 这是一个简单的练习，让你对 SSD 的可行性有一些有趣的了解。回想一下对于磁盘， $1\text{PB} = 10^9$ MB。那么下面对单位的直接翻译得到了下面的每种情况的预测时间：

A. 最糟糕情况顺序写(470MB/s)： $(10^9 \times 128) \times (1/470) \times (1/(86\,400 \times 365)) \approx 8$ 年。